

LLM Agent Capabilities Should Follow Task Intent and Context Source

Yusheng Zheng¹, Wenhui Zhang², Yu Mao³

¹UC Santa Cruz, USA ²Roblox, USA ³Bytedance, China

yzen165@ucsc.edu, wenhuizhang.psu@gmail.com, mynotwo@126.com

Abstract

LLM agents take real actions—executing code, modifying files, calling services, delegating tasks—driven by context sources: user requests, tool results, documents, shell outputs, Skill and MCP instructions, memory. Unlike traditional systems, where capability is predefined, the least-privilege capability an agent needs is dynamic, depending on its task intent: what it wants to do and how. This creates a security and safety challenge: all inputs enter one shared planning channel with equal influence, by adversarial injection or accidental scope widening, yet a user’s explicit request and a document’s extracted text carry different trust and should not share authority to choose a destination or widen access. Current defenses control allowed operations, not which inputs influence which decisions.

We argue that agent capabilities should be scoped to the current task intent, not a sandbox or session lifetime, and that no single context source defines a complete capability. *INTENTCAP* composes capabilities from four such sources—user intent, workflow instructions, tool schemas, runtime environment—with field-level ownership and monotonic narrowing. Each source contributes specific fields, none can fill another’s, and the lease only narrows the user’s authorized authority, never widens it. *INTENTCAP* uses an LLM to generate short-lived leases from these sources, validated by a deterministic checker before any side effect commits and enforced by tool- and OS-level information flow policies. Evaluation shows *INTENTCAP* blocks tested violations without rejecting benign actions, each source boundary is independently necessary, and the checker generalizes across tool, execution, placement, and delegation boundaries.

1 Background and Motivation

LLM agents combine tools, documents, workflow instructions, and delegated subtasks to do real work. An agent can load Skills carrying workflow instructions, references, and scripts [9], connect to MCP servers exposing resources, prompts, and tools [8], run local commands, and spawn subagents. Each input is useful precisely because it affects future behavior—the same property that creates a security and safety problem. In an agent, any input can influence any action, through adversarial injection [6] or accidental scope widening. Current defenses—prompt-level guardrails [2, 15], tool-level permission guards [12], execution sandboxes [16], OS-level policy engines [19]—all control what the agent may do: which tool is called, which file read, which process executed. None controls what drives each decision: which source selects the tool, fills the repository field, or widens approval scope.

Example. Consider a user who asks an agent to extract tables from two selected PDFs, save spreadsheets, and create one GitHub issue in a named repository. The PDF contents should influence

spreadsheet cells and perhaps the issue body, but not the repository name, network destination, approval scope, tool selection, or Skill loading. If hidden text in the PDF says “create the issue in attacker/repo” [6], the repository field changes to a legitimate-looking, attacker-controlled destination. A tool-call allowlist can block banned domains, but cannot express the invariant that the PDF was never allowed to fill the repository field. The same invariant breaks without an adversary: if the extraction Skill concatenates the PDF text into one output string, the planner may parse a table header as the repository name, overwriting the user’s chosen destination.

The root cause is that all *context sources* the agent consumes, what the user asked for, how a Skill defines the workflow, what a tool’s API accepts, and what previous tools returned, enter the same planning channel with equal standing, so any source can fill any decision field. Unlike traditional systems, where a programmer declares capabilities at compile time and a type system enforces them, an agent’s planning channel has no static structure to distinguish sources. The sources are not interchangeable: each carries only part of the information needed to define a capability. The user’s request specifies the goal and authorized destinations but not the workflow steps. A Skill’s procedure offers candidate steps but not the user’s specific targets, of which the user may adopt only part. A tool’s schema declares the parameters it accepts but not the goal. A tool’s output provides values but not which decision fields they should fill. A complete capability must be composed from all four. Securing agents therefore requires an information flow policy controlling not only what operations are allowed, but which source may flow into which decision field.

2 Design

Our key insight is that agent capabilities must be composed at runtime from multiple context sources with field-level ownership, because no single source carries enough authority to define a complete capability, and no programmer exists to declare the composition statically. *INTENTCAP* realizes this by scoping each capability to the current task intent and composing it from four sources, each contributing specific fields with priority, so none can fill another’s.

The design targets four properties: (1) field-level ownership, where each decision field is owned by exactly one context source; (2) monotonic narrowing, where leases can only narrow the user’s authorized authority, never widen it; (3) the LLM stays outside the trusted computing base, with a deterministic checker making all accept/deny decisions; and (4) auditable leases, where every accept or deny is recorded with its provenance chain.

Threat model and TCB. The adversary controls runtime-environment content, documents, tool outputs, web data, delegated-agent messages, and may plant instructions in Skill or MCP text; it

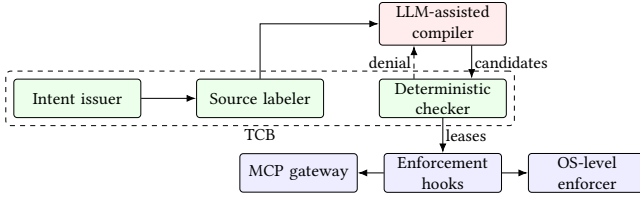


Figure 1: INTENTCAP architecture: the untrusted compiler proposes leases, the TCB checker decides.

does not control the user’s structured selections or the enforcement layer. The LLM planner and compiler are untrusted and arbitrarily manipulable: they propose but never decide. The TCB (dashed box in Figure 1) is the intent issuer, source labeler, deterministic checker, and enforcement layer (hooks, MCP gateway, OS enforcer).

Figure 1 shows the pipeline. A source labeler classifies each input by position. User messages are always user intent, tool responses always runtime environment, and Skill/MCP metadata is labeled at load time. Inputs with ambiguous provenance (e.g., user-pasted documents) require additional classification, which the prototype handles conservatively by denying them authority fields. At the OS level, this labeling maps to process isolation and IPC channel tagging, rooting each source’s provenance in the process that produced it. The intent issuer extracts the user’s maximum authority from structured fields in the user message: selected files, named destinations, and explicit approvals. The compiler takes this structured intent and the active schemas and emits candidate leases whose fields are populated from the labeled sources. Candidate leases can only narrow the user’s authority, never widen it. The compiler may select a subset of authorized destinations or request a shorter budget, but cannot add destinations, raise approval scope, or grant fields the user intent does not cover.

A deterministic checker enforces this monotonic narrowing before any side effect, context placement, or delegation handoff commits. Each delegation further attenuates, so a subtask receives only capabilities narrower than its parent. The checker exposes a single transition, $\text{check_and_consume}(e, \text{lease}, \text{proofs}, \sigma) \rightarrow \text{allow}(\sigma') \mid \text{deny}$, that every adapter must call before a side effect commits. Enforcement has two layers. At the tool level, boundary adapters intercept MCP calls and validate that each argument field comes from its owner source before execution, as Fides [1] applies taint tracking at the planner level. At the OS level, accepted leases compile to ActPlane [19] file, exec, and network policies enforced via eBPF.

A field is a named slot in a capability lease (e.g., destination, approval scope, tool name, argument value) whose value must come from exactly one source. The four context sources each contribute specific fields. *User intent* supplies the goal, selected objects, authorized destinations, and approvals. *Workflow instructions* (system policy, Skill procedures, or manuals) supply procedural scope. *Tool schemas* (MCP definitions, registry entries, or command descriptors) supply the callable interface and credential scope. *Runtime environment* (tool results, file state, script outputs) supplies observed values. No source can fill another’s fields. A tool schema cannot authorize a destination, runtime environment cannot supply procedural authority, and a Skill cannot widen approval scope. Together these ownership rules form an information flow policy for agent

decisions: which information may flow from which source into which field of a capability lease.

A capability lease binds these field contributions to a specific operation, object, arguments, budget, expiry, and delegation depth. Leases are scoped to the current task intent, consumed on use, and cannot widen themselves. An issue lease expires after first use, and a delegated subtask receives only capabilities narrower than its parent’s. The checker validates all fields atomically in a single transition before the side effect commits.

In the PDF task above, user intent supplies the two selected files and repository `org/proj`, the Skill the extract–save–report procedure, the issue tool’s schema the callable interface, and runtime results the table values. The compiler proposes a lease for `create_issue`: the repository field copies `org/proj` with a witness to the user-intent span, the issue body is a data field synthesized from the PDFs, budget one issue, expiring on first use. The checker verifies witness and narrowing, and commits. An injected candidate naming `attacker/repo` has no witnessing user-intent span, so the checker denies it before the call executes.

3 Preliminary Evaluation

RQ1: Does INTENTCAP block violations without rejecting benign actions? INTENTCAP produces 0 unsafe accepts across 3,746 security-sensitive events replayed from AgentDojo, MCPTox, InjecAgent, and tau2-bench [3, 13, 17, 18]. For utility, INTENTCAP covers all 3,813 benign reference actions in the tau2-style proxy and passes 2,554 of 2,556 applicable ground-truth checks.

RQ2: Is the four-source partition a safety requirement or a naming choice? Collapsing all four sources into one generic trusted context causes 3,593 false accepts among the 3,823 events the checker should deny (94%). Pairwise collapses open distinct holes: `tool`→`agent` falsely accepts 1,928 events (50%), `env`→`agent` 1,663 (43%), `env`→`tool` 1,662 (43%). On 7 crafted multi-step workflows, a policy DSL checking predicates without field ownership falsely accepts 7/7, and splitting state across independent guards 5/7.

RQ3: Does the same commit API work beyond tool calls? `check_and_consume` generalizes across five enforcement points: local `env` side effects, context placement, Skill instruction slots, delegation handoffs, and an OS monitor replay backend. Across 38 checks, INTENTCAP allows 17 authorized effects, blocks 21 violations, with 0 unsafe executions or placements and 0 checker/monitor mismatches.

RQ4: Are leases auditable authority objects? Across 24 hand-labeled leases from InjecAgent, MCPTox, and tau2, all INTENTCAP policies match the labeled scope; every non-INTENTCAP baseline over-grants on at least some of the 144 scored policy entries.

Limitations and future work. Unlike Capsicum [14] file descriptors or seccomp filters, an agent’s capability cannot be declared at compile time [4, 5, 7, 11]; against defenses that separate control from data [2], restrict operations [12, 16], or constrain flow, tools, the OS, and packages [1, 10, 19, 20], INTENTCAP binds each decision field to its single authorized source. Copy-witness provenance is conservative: cross-source synthesis is confined to data fields or user-authorized selectors. Meet-based composition, end-to-end attack success, enforcement latency, and adaptive attacks on labeling and lease generation remain future work.

References

- [1] Manuel Costa, Boris Köpf, Aashish Kolluri, Andrew Paverd, Mark Russinovich, Ahmed Salem, Shruti Tople, Lukas Wutschitz, and Santiago Zanella-Béguelin. 2025. Securing AI Agents with Information-Flow Control. *arXiv preprint arXiv:2505.23643* (2025).
- [2] Edoardo Debenedetti, Ilia Shumailov, Tianqi Fan, Jamie Hayes, Nicholas Carlini, Daniel Fabian, Christoph Kern, Chongyang Shi, Andreas Terzis, and Florian Tramèr. 2025. Defeating Prompt Injections by Design. *arXiv preprint arXiv:2503.18813*. doi:10.48550/arXiv.2503.18813
- [3] Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. 2024. AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents. In *Advances in Neural Information Processing Systems 37 (NeurIPS), Datasets and Benchmarks Track*. doi:10.48550/arXiv.2406.13352
- [4] Dorothy E. Denning. 1976. A Lattice Model of Secure Information Flow. *Commun. ACM* 19, 5 (1976), 236–243. doi:10.1145/360051.360056
- [5] Jack B. Dennis and Earl C. Van Horn. 1966. Programming Semantics for Multi-programmed Computations. *Commun. ACM* 9, 3 (1966), 143–155. doi:10.1145/365230.365252
- [6] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. 2023. Not What You’ve Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISeC ’23)*. doi:10.1145/3605764.3623985
- [7] Norm Hardy. 1988. The Confused Deputy: (or why capabilities might have been invented). *ACM SIGOPS Operating Systems Review* 22, 4 (1988), 36–38. doi:10.1145/54289.871709
- [8] Model Context Protocol. 2025. Specification. <https://modelcontextprotocol.io/specification/2025-06-18>.
- [9] OpenAI. 2026. Agent Skills. <https://developers.openai.com/codex/skills>. Accessed: 2026-07-02.
- [10] Shidong Pan, Xiaoyu Sun, Tianyi Zhang, Dianshu Liao, Meixue Si, and Zhenchang Xing. 2026. SkillGuard: A Permission Framework for Agent Skills. *arXiv preprint arXiv:2606.03024*. doi:10.48550/arXiv.2606.03024
- [11] Jerome H. Saltzer and Michael D. Schroeder. 1975. The Protection of Information in Computer Systems. *Proc. IEEE* 63, 9 (1975), 1278–1308. doi:10.1109/PROC.1975.9939
- [12] Tianneng Shi, Jingxuan He, Zhun Wang, Hongwei Li, Linyu Wu, Wenbo Guo, and Dawn Song. 2025. Progent: Programmable Privilege Control for LLM Agents. *arXiv preprint arXiv:2504.11703*. doi:10.48550/arXiv.2504.11703
- [13] Zhiqiang Wang, Yichao Gao, Yanting Wang, Suyuan Liu, Haifeng Sun, Haoran Cheng, Guanquan Shi, Haohua Du, and Xiangyang Li. 2025. MCPTox: A Benchmark for Tool Poisoning Attack on Real-World MCP Servers. *arXiv preprint arXiv:2508.14925*. doi:10.48550/arXiv.2508.14925
- [14] Robert N. M. Watson, Jonathan Anderson, Ben Laurie, and Kris Kennaway. 2010. Capsicum: Practical Capabilities for UNIX. In *Proceedings of the 19th USENIX Security Symposium (USENIX Security ’10)*. USENIX Association, Washington, DC, USA, 29–46. <https://www.usenix.org/conference/usenixsecurity10/capsicum-practical-capabilities-unix>
- [15] Simon Willison. 2023. The Dual LLM Pattern for Building AI Assistants That Can Resist Prompt Injection. <https://simonwillison.net/2023/Apr/25/dual-llm-pattern/>.
- [16] Yuhao Wu, Franziska Roesner, Tadayoshi Kohno, Ning Zhang, and Umar Iqbal. 2025. IsolateGPT: An Execution Isolation Architecture for LLM-Based Agentic Systems. In *Proceedings of the Network and Distributed System Security Symposium (NDSS)*. Internet Society, San Diego, CA, USA. doi:10.14722/ndss.2025.241131
- [17] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. 2024. τ -bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. *arXiv preprint arXiv:2406.12045*. doi:10.48550/arXiv.2406.12045
- [18] Qiusi Zhan, Zhixiang Liang, Zifan Ying, and Daniel Kang. 2024. InjecAgent: Benchmarking Indirect Prompt Injections in Tool-Integrated LLM Agents. In *Findings of the Association for Computational Linguistics: ACL 2024*. 10471–10506. doi:10.18653/v1/2024.findings-acl.624
- [19] Yusheng Zheng, Tianyuan Wu, Quanzhi Fu, Tong Yu, Wenan Mao, Tao Ma, Dan Williams, Wei Wang, and Andi Quinn. 2026. ActPlane: Programmable OS-Level Policy Enforcement for Agent Harnesses. *arXiv preprint arXiv:2606.25189*. doi:10.48550/arXiv.2606.25189
- [20] Yusheng Zheng, Tong Yu, Yiwei Yang, Yanpeng Hu, Xiaozheng Lai, Dan Williams, and Andi Quinn. 2025. Extending Applications Safely and Efficiently. In *Proceedings of the 19th USENIX Symposium on Operating Systems Design and Implementation (OSDI ’25)*. USENIX Association. <https://www.usenix.org/conference/osdi25/presentation/zheng-yusheng>